Flutter Developer Assignment

Platform Commons · Engineering Take-Home · Estimated effort: 12-15 hrs

Before You Start

This assignment describes the bare minimum we expect.

If you know a better way to structure the code, a slicker animation, or a smarter approach to anything here — just build it. We are not checking boxes. We want to see how you think and what you are capable of. The best submissions **always go beyond** what is written here.

What You Are Building

Build a movie discovery app where multiple users can each save movies they want to watch. The app then shows which movies more than one user has saved — making it easy for a group to agree on what to watch.

The app has six pages: a Users page, an Add User form, a Movies page, a Movie Detail page, a Saved Movies page per user, and a Matches page.

Everything must work without the internet. A user can be added offline, save movies offline, and have everything sync to the server automatically when the internet comes back — with no data lost and no duplicates.

PAGE 01

Users Page

The first page the user sees when they open the app.

- Show a scrollable list of users fetched from the Reqres API. Load the next page automatically when the user scrolls to the bottom.
- Each list item shows the user's **profile photo**, **first name**, **last name**, and **how many movies they have saved**.
- A button (floating action button or similar) opens the Add User form (Page 02).
- Tapping a user opens the Movies page (Page 03).
- A **Matches** button — always visible in the app bar or nav bar — opens the Matches page (Page 06).
- While the list is loading, show a **skeleton / shimmer placeholder** instead of a blank screen or spinner.

Endpoint

```
GET https://reqres.in/api/users?page={page}
```

Get your Reqres API key from reqres.in and include it in every request header.

PAGE 02

Add User

A form to create a new user. Must work with or without internet.

- The form has two fields: **Name** and **Movie Taste** (a short note like "loves horror" or "no sad endings"). Movie Taste maps to the `job` field in the Reqres API.
- **If online:** Save the user to the local database and POST the new user to the API immediately when the form is submitted.

Endpoint

```
POST https://reqres.in/api/users
Body: { "name": "Alex", "job": "loves horror" }
```

- **If offline:** Save the user to the local database immediately with a `pending_sync = true` flag.
- When the internet comes back, WorkManager sends the pending user to the server in the background and saves the server-assigned ID back to the local record.
- The new user appears in the Users page and can save movies right away — the app does not wait for sync before showing them.

PAGE 03

Movies Page

A browsing page where a user can scroll through trending movies and save the ones they want to watch.

- Show a scrollable, paginated list of trending movies from the TMDB API. Load more as the user scrolls.
- Each movie card shows the **poster image**, **title**, and **release year**.
- Each card shows a **count badge** — how many users have saved this movie. This number updates in real time as users save or unsave movies.
- A **Save** button on each card lets the currently active user (whoever was tapped on Page 01) save or unsave that movie.
- An action button in AppBar for navigating to User's Saved Movies (Page 05).
- Tapping the card itself (not the Save button) opens the Movie Detail page (Page 04).
- **When a movie is fetched from the API, save its data to the local database.** This is required so the Saved Movies page (Page 05) works without internet.

Endpoint (TMDB)

```
GET https://api.themoviedb.org/3/trending/movie/day    ?language=en-
US&page={page}&api_key=YOUR_KEY
```

Backup API if TMDB is unavailable:

Movie list (OMDB)

```
GET https://www.omdbapi.com/?s={query}&page=2&apikey=YOUR_KEY
```

Movie detail (OMDB)

```
GET https://www.omdbapi.com/?apikey=YOUR_KEY&i={imdbId}
```

PAGE 04

Movie Detail Page

A detail view for a single movie. Opens from the Movies page or the Saved Movies page.

- Show the movie's **title**, **description**, **release date**, and a **large poster image**.
- A **Save** button lets the active user save or unsave the movie.
- Show a line like **"2 users want to watch this"** with small profile photos of those users. If no one has saved it yet, show "Be the first to save this."
- Use `http://image.tmdb.org/t/p/w185/{poster_path}` for small images and `w500` for the large image on this page.

Endpoint

```
GET https://api.themoviedb.org/3/movie/{movie_id}?api_key=YOUR_KEY
```

PAGE 05

User's Saved Movies

Shows all movies a specific user has saved. Opens by tapping a user on the Users page.

- Show the user's **profile photo**, **name**, and **Movie Taste** note at the top.
- Below, list all their saved movies with poster and title.
- The currently active user can save a movie directly from this page.
- If the user has no saved movies, show an empty state with a prompt to go browse the Movies page.
- **This page must work fully offline.** Movie data is stored locally when fetched (Page 03), so saved lists are always readable with no internet.

PAGE 06

Matches Page

Shows movies that more than one user has saved.

- List every movie saved by **2 or more users**, sorted from most saves to least.
- Each row shows the **poster**, **title**, **save count**, and small profile photos of the users who saved it.
- If all users have saved the same movie, highlight it visually as the top pick.
- If there are no matches yet, show an empty state explaining how matches work.
- Tapping a movie opens the Movie Detail page.
- **This page reads entirely from the local database** — no API call needed. Use a **Stream** so the list updates automatically whenever a user saves or removes a movie.

PAGE 07

Offline & Sync Behaviour

This is not a visible page — it describes how the app must behave when there is no internet or the connection is slow.

Offline test we will run:

We will turn on airplane mode and do all of the following without any internet:

- Add a new user
- Open that user's Saved Movies page (should be empty but visible)
- Go to the Movies page (should show movies that were already loaded)
- Save a few movies for that user
- Turn the internet back on
- Confirm the new user has synced to the server with the correct ID, and their saved movies are still correctly linked to them

Nothing should crash. No data should be lost. No duplicate users or saved movies should appear.

Bad connection test we will run:

We have added a setting in the app that randomly blocks 30% of network calls to simulate a weak connection. Your app must:

- Automatically retry any failed network call, waiting a little longer between each retry
- Never crash or show a full-screen error for a list that was already loaded
- Show a small **"reconnecting..."** bar while a retry is in progress — not a blocking pop-up

The user should barely notice anything went wrong. The app should fix itself.

PAGE 08

UI & Design

This matters as much as the functionality.
We want to see a UI that feels smooth and well made — not just one that works.

Pick **one** design system and use it consistently on every page:

- **Material Design 3** — Flutter's built-in modern design system with updated components and colour theming
- **Cupertino** — Flutter's iOS-style components
- **Your own** — define a set of colours, font sizes, spacing values, and corner sizes and apply them everywhere. Explain your choices in the README.

What we will check:

- Navigating between pages feels smooth — the movie poster should animate from the list card into the detail page (hero animation)
- List items should fade in one by one when a page first loads — not all appear at once
- Show a skeleton / shimmer while content is loading, not a plain spinner
- Images should fade in gently as they load
- Scrolling should stay smooth, even on long lists
- Buttons and inputs should give clear feedback when tapped
- The save count badge should animate when its number changes
- After saving a movie, adding a user, or completing a sync — show a small snackbar at the bottom. No blocking dialogs.

PAGE 09

If You Used AI

Using AI tools is allowed. We just ask that you be transparent.
We don't care whether you used it. We want to see how well you directed it.

If you used any AI tool (ChatGPT, Claude, Gemini, Copilot, Cursor, v0, or anything else), include one of the following with your submission:

- **A prompt document** — a PDF or Markdown file showing every prompt you sent, in order, with enough context to understand why you wrote each one
- **A public conversation link** — a shareable link to the full AI conversation (ChatGPT share link, Claude.ai share link, etc.)

No penalty for using AI. We only mark down submissions that used it without disclosing it.

Technical Checklist

What we check	How to build it
Language	Dart
App state management	Bloc, Provider, Riverpod or Cubit
Dependency injection	get_it or Riverpod
Network calls	Dio or http package
Local database	Drift (recommended) or any SQLite library
Background sync	WorkManager — runs automatically when internet returns
Pagination	Infinite scroll or PagedListView
Live data (Matches page)	Streams from the local database
Image loading	CachedNetworkImage with fade-in
UI design system	Material 3, Cupertino, or your own

API Key Setup

TMDB

- Sign up at themoviedb.org → Settings → API → request a developer key.
- Replace `YOUR_KEY` in all TMDB URLs with your key.

OMDB (backup — use only if TMDB is unavailable)

- Get a free key at omdbapi.com and replace `YOUR_KEY` in all OMDB URLs.

What to Submit

- A link to a **public Git repository** with your source code and a working **APK**
- A **README** that covers: what the app does, how the code is structured, which design system you used, how the local database tables are set up (the user → saved movies relationship is worth explaining), and any decisions that are not obvious from the code
- If you used AI: your prompt document or conversation link (see Page 09)

Scoring

Area	Weight	What we look at
All pages working correctly	30%	Every page functions as described; the full offline flow works end to end
UI quality	20%	Consistent design, smooth scrolling, animations, skeleton screens

Code quality	20%	Clean structure, easy to read, well named, properly organised
Offline sync	15%	Add user offline → save movies → come back online → nothing lost
Bad connection handling	10%	Retries quietly, no crashes, no duplicate data
AI prompting (if used)	5%	Prompts show clear thinking and useful context

We look forward to reviewing your work. If anything is unclear, feel free to reach out.

Platform Commons · Engineering Team